Artificial Intelligence

Lecture Note: icyice

Illustration: ©SEGA ©Colorful Palette
©Crypton Future Media

Contents

1 Basic Algorithms of AI **1**

§1.1 Search 1

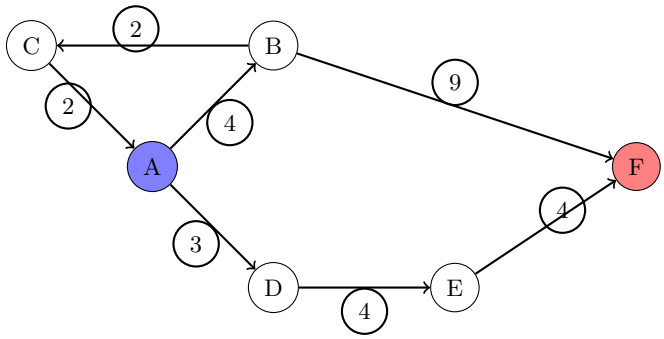
1 Basic Algorithms of AI

§1.1 Search

A search problem consists of:

- States
- Initial state
- Goal test
- Actions
- Transition model
- Path cost

Example 1.1



- State space:
  - Vertices
- Initial state:
  - A
- Goal test:
  - Is state = F?
- Actions:
  - Go to adjacent vertex with cost = weight
- Transition model
- Path cost
  - cost = weight
- Solution: A → D → E → F

Definition1.1 State Graphs

State space graph: A mathematical representation of a search problem ♣

- **Nodes** are (abstracted) world configurations(**states**)
- **Arcs** represent successors (action results)
- **The goal test** is a set of goal nodes (maybe only one)

Notation:

In a state graph, each state occurs only once.

Definition1.2 Search Tree

A "what if" tree of plans and their outcomes. ♣

- The start state is the root node
- Children nodes correspond to successors
- Nodes show states, but correspond to PLANS that achieve those states

Notation:

For most of the problems, we can never build the whole tree. The search tree does not remember whether the node has occurred or not, so it may have infinite outcomes.

To avoid redundant paths, we could mark the node that has occurred (because the following tree is repeated).

Summary:

Algorithm 1 Tree Search

```
function TREE-SEARCH(problem) returns a solution, or failure
initialize the frontier using the initial state of problem
loop do
    if the frontier is empty then return failure
    choose a leaf node and remove it from the frontier
    if the node contains a goal state then return the corresponding solution
    expand the chosen node, adding the resulting nodes to the frontier
```

Algorithm 2 Graph Search

```
function GRAPH-SEARCH(problem) returns a solution, or failure
initialize the frontier using the initial state of problem
initialize the explored set to be empty
loop do
    if the frontier is empty then return failure
    choose a leaf node and remove it from the frontier
    if the node contains a goal state then return the corresponding solution
    add the node to the explored set
    expand the chosen node, adding the resulting nodes to the frontier
        only if not in the frontier or explored set
```

Annotation:

Some important ideas:

- **Frontier**—Nodes to be explore

- **Expansion**—Add the children nodes to the current node
- **Explored set**—The set of nodes that have been expanded
- **Exploration strategy**—which leaf nodes to explore first

Performance measurements:

- **Completeness**: Guaranteed to find a solution if one exists?
- **Optimality**: Guaranteed to find the optimal solution?
- **Time complexity**: How long?
- **Space complexity**: How much momery?

Annotation:

Search strategies:

- **Uninformed search**(blind search): no additional information about states
- **Informed search**(heuristic search): knows how "promising" a non-goal state is

Definition1.3 Breadth First Search

- **Strategy**:
  - **Expand a shallowest node first**
- **Implementation**:
  - **Frontier is a FIFO (first in first out) queue**

♣

Notation:

BFS will expand the search tree layer by layer, so for the worst conditions, the time and space complexities are both $O(b^d)$.

Annotation:

How to optimize breadth first search:

- **Cost-Sensitive Search**
  - To find **the optimal solution** with any step cost function
- **Strategy**:
  - Expand a **cheapest** node first
- **Implementation**:
  - Frontier is a **priority queue**

Definition1.4 Depth First Search

- **Strategy**:
  - **Expand a deepest node first**
- **Implementation**:
  - **Frontier is a LIFO (last in first out) stack**

♣

Notation:

If the depth is infinite, DFS might not find the solution, but for tree search, the space complexity is $O(bm)$ (put every children node into the stack).

Iterative Deepening DFS:

- **Idea: get DFS's space advantage with BFS's time / shallow-solution advantages**
  - Run a DFS with depth limit 1. If no solution...
  - Run a DFS with depth limit 2. If no solution...
  - ...
  - Run a DFS with depth limit d. ...

Annotation:

Most of the problem works in the low levels, so not so bad to be redundant.

Summary:

Uninformed Search Strategies:

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening
Complete?	Yes^a	Yes^{a,b}	No	No	Yes^a
Time	$O(b^d)$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	$O(b^m)$	$O(b^\ell)$	$O(b^d)$
Space	$O(b^d)$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	$O(bm)$	$O(b\ell)$	$O(bd)$
Optimal?	Yes^c	Yes	No	No	Yes^c

If no any other information is offered, there's no search algorithm that is not exponential.

Definition1.5 Search Heuristics

A heuristic function is:

- A function that estimates how close a state is to a goal
- Designed for a particular search problem
- Examples: Manhattan distance, Euclidean distance for pathing

♣

Definition1.6 Greedy Best-First Search

Strategy:

- Expand a node with lowest $h(n)$ first

♣

Notation:

Greedy Best-First Search is incomplete, if $h(n)$ is smaller but there is no solution in that path, the algorithm will end up in loops.

Definition1.7 A* Search

Strategy:

- Let $g(n)$ be cost incurred already on path to n
- Expand nodes with lowest $g(n) + h(n)$ first

♣

If the heuristic is **admissible**, the **tree-search** version of A* is optimal

- A heuristic $h(n)$ is **admissible** if

$$0 \leq h(n) \leq h^*(n)$$

  where $h^*(n)$ is the true cost to a nearest goal
- Example: straight-line distance is admissible
- Admissible heuristic means that A* is always optimistic

Notation:

For graph search, the heuristic needs to be consistent.

A heuristic $h(n)$ is **consistent (monotonicity)** if for every node n and every successor n' of n generated by any action a,

$$h(n) \leq c(n, a, n') + h(n')$$

It is a form of the general triangle inequality.

Annotation:

- If the heuristic is **admissible**, the **tree-search** version of A* is optimal
- If the heuristic is **consistent**, the **graph-search** version of A* is optimal

  Which means A* is complete and optimally efficient.

Example 1.2

Design a heuristic function for the puzzle below

- **Initial state:**

1	2	3
8		4
7	6	5

- **Goal state:**

1	2	3
4	5	6
7	8	

solve. There are two common solutions.

- h_1 = the number of tiles misplaced
- h_2 = the sum of distances of the tiles from their goal positions